FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos II

Unidad 2 / Semana 4

Año: 2025

Contenidos

Comunicación entre procesos: Procesos y multiprogramación. Procesos concurrentes. Cooperación e Interferencia.

Índice

1. Procesos y multiprogramación **3**
2. Procesos concurrentes **3**
2.1 Formas de ejecutar procesos concurrentes 4
2.2 Concurrencia vs. Paralelismo 5
3. Comunicación entre procesos (IPC) **6**
3.1 Importancia de la comunicación entre procesos 7
3.2 IPC a través de tuberías 7
3.3 Memoria compartida (Shared Memory) 9
3.4 Pasaje de mensajes (message passing) 11
3.5 Cuadro comparativo entre los mecanismos pasaje de mensaje y memoria compartida 13
4. Cooperación e interferencia entre procesos **13**
4.1 Cooperación entre procesos 13
4.2 Interferencia entre procesos 14

1. Procesos y multiprogramación

Uno de los objetivos de los sistemas operativos es la **multiprogramación**, es decir, admitir varios **procesos** en memoria para maximizar el uso del procesador. Esto funciona ya que los procesos se irán intercambiando el uso del procesador para su ejecución de forma **concurrente**. Para ello, el sistema operativo organiza los procesos en varias colas pasándolos de unas colas a otras.

Es importante diferenciar multiprogramación o multitarea de multiprocesamiento:

Multiprogramación es un cuando existe un único procesador.

En este modelo todos los procesos concurrentes se ejecutan sobre un único procesador. El sistema operativo se encarga de ir repartiendo el tiempo del procesador entre los distintos procesos, intercalando la ejecución de los mismos para dar así una apariencia de ejecución simultánea.

Multiprocesamiento se da cuando existen varios procesadores o núcleos.

Un multiprocesador es una máquina formada por un conjunto de procesadores que comparten memoria principal. En este tipo de arquitecturas, los procesos concurrentes no solo pueden intercalar su ejecución sino también superponerla. En este caso si existe una verdadera ejecución simultánea de procesos, al coincidir las fases de procesamiento de distintos procesos. En un instante dado se pueden ejecutar de forma simultánea tantos procesos como procesadores haya.

2. Procesos concurrentes

Los procesos concurrentes son aquellos que, aunque parezcan estar ocurriendo al mismo tiempo, no necesariamente lo están en un instante físico. La clave está en que su ejecución se superpone en el tiempo, lo que puede dar la ilusión de simultaneidad. Esta propiedad es fundamental en los sistemas operativos modernos y en la programación, ya que permite que múltiples tareas se realicen de manera eficiente.

Debemos diferenciar entre la simultaneidad real y la aparente.

- La **simultaneidad real** ocurre en sistemas con múltiples procesadores (multiprocesadores o multicore), donde cada proceso se ejecuta en un núcleo distinto. En este escenario, dos o más procesos están literalmente activos en el mismo momento.
- La **simultaneidad aparente** se da en sistemas con un solo procesador. Aquí, el sistema operativo utiliza técnicas como el cambio de contexto (context switching) para alternar rápidamente entre la ejecución de diferentes procesos. Por ejemplo, el procesador ejecuta una porción del proceso A, luego cambia al proceso B, y así sucesivamente. Para el usuario, esto se ve como si el navegador, el reproductor de música y el editor de texto estuvieran funcionando al mismo tiempo.

2.1 Formas de ejecutar procesos concurrentes

Existen tres modelos de **concurrencia** en sistemas operativos, que se diferencian por cómo se distribuyen y ejecutan las tareas (representadas por círculos) y los procesos (representados por cuadrados) en los procesadores (representados por cuadros con textura) disponibles.

Multiprogramación

En la **multiprogramación**, un solo procesador gestiona múltiples procesos. La CPU no ejecuta todos los procesos a la vez, sino que el sistema operativo los **intercala** rápidamente. Cuando un proceso está esperando una operación de E/S, el sistema operativo le cede el control a otro proceso. Esto crea la ilusión de que todos los procesos se ejecutan simultáneamente, lo que aumenta la **utilización del CPU**.

Multiprocesamiento

El modelo de **multiprocesamiento** implica que las tareas se dividen entre **múltiples procesadores** que operan dentro de una única máquina. Cada procesador puede ejecutar un proceso o una tarea de forma **paralela**. En este caso, la concurrencia es real: las tareas se ejecutan al mismo tiempo en diferentes

procesadores, lo que aumenta la velocidad de ejecución y el rendimiento del sistema en general.

Procesamiento Distribuido

El **procesamiento distribuido** es el modelo más complejo. Aquí, las tareas y los procesos se distribuyen y ejecutan en **múltiples computadoras** que están interconectadas a través de una red. Cada máquina tiene sus propios procesadores y memoria, y se comunican entre sí para completar la tarea. Este modelo ofrece una alta escalabilidad y tolerancia a fallos, ya que si una máquina falla, las otras pueden seguir funcionando.

2.2 Concurrencia vs. Paralelismo

La concurrencia indica paralelismo potencial, esto se debe a que la concurrencia es una condición necesaria pero no suficiente para el paralelismo real.

Veamos la diferencia entre ambos conceptos:

- **Concurrencia:** Es una propiedad del diseño de un sistema. Se trata de cómo se estructuran las tareas para que se puedan superponer en el tiempo. Por ejemplo, en un sistema de un solo núcleo, el sistema operativo intercala la ejecución de múltiples procesos, lo que crea la **apariencia de simultaneidad** aunque solo una tarea se esté ejecutando en un instante dado. Es una forma de manejar varias tareas en progreso al mismo tiempo, sin importar si hay uno o más procesadores.
- **Paralelismo:** Es la ejecución real de múltiples tareas al mismo tiempo. Esto solo es posible en sistemas con **múltiples núcleos o procesadores**. Cada tarea se ejecuta de forma independiente en su propio procesador. El paralelismo es una forma específica de concurrencia que requiere hardware dedicado.

En resumen, la concurrencia es el **paradigma** para manejar múltiples tareas, mientras que el paralelismo es la **ejecución física** de esas tareas al mismo tiempo. Un sistema concurrente puede no ser paralelo si solo tiene un procesador, pero si tiene varios, entonces tiene el **potencial** de ejecutar las tareas de forma paralela.

3. Comunicación entre procesos (IPC)

La comunicación entre procesos (**IPC**, por sus siglas en inglés, *Inter-Process Communication*) es un conjunto de mecanismos que un sistema operativo provee para permitir que los **procesos intercambien información**, se **sincronicen** y **coordinen** sus actividades. En un sistema **multiprogramado**, donde múltiples procesos pueden estar en ejecución al mismo tiempo (de manera concurrente), la IPC es fundamental para que estos puedan cooperar y compartir recursos. Sin estos mecanismos, cada proceso operaría de manera aislada, sin la capacidad de interactuar con otros.

La forma en que los procesos concurrentes interactúan entre sí se clasifica en dos categorías principales: **independientes** y **cooperativos**.

- **Procesos Independientes**: Estos procesos no tienen ninguna relación de comunicación o dependencia entre sí. No afectan ni son afectados por otros procesos en el sistema. Su principal interacción es la **competencia por recursos escasos**. Imagina un sistema con una sola impresora y dos procesos que quieren imprimir documentos. Ambos procesos compiten por el uso de la impresora. El sistema operativo debe decidir qué proceso obtiene acceso al recurso y cuándo, sin que uno afecte el resultado del otro. Esta competencia requiere mecanismos de gestión de recursos para evitar conflictos y asegurar que todos los procesos puedan ejecutarse.

- **Procesos Cooperativos**: Los procesos cooperativos están diseñados para trabajar juntos hacia un objetivo común. A diferencia de los procesos independientes, estos procesos pueden **afectar o ser afectados por otros** porque **comparten datos o recursos**.
  La **comunicación entre procesos (IPC)** es fundamental para los procesos cooperativos, permitiendo el intercambio de información y la sincronización. Esto permite que múltiples procesos puedan coexistir y colaborar de manera eficiente.

3.1 Importancia de la comunicación entre procesos

La necesidad de la **comunicación entre procesos (IPC)** es fundamental para el funcionamiento coordinado de los sistemas multiprogramados, ya que **facilita la interacción y colaboración entre procesos, logrando objetivos como**:

- **Cooperación:** Una tarea compleja se divide en subtareas que se ejecutan como procesos separados. Por ejemplo, en un servidor web, un proceso podría escuchar las peticiones entrantes mientras otro se encarga de procesar cada petición.
- **Compartición de datos:** Los procesos a menudo necesitan compartir datos para realizar sus funciones. La IPC facilita esta compartición de una manera controlada, evitando la corrupción de datos.
- **Modularidad:** Al dividir un sistema en módulos independientes (procesos), se simplifica el desarrollo y mantenimiento del software. La IPC permite que estos módulos se comuniquen y actúen como un sistema unificado.
- **Coordinación y sincronización:** Los procesos necesitan coordinar sus acciones para evitar conflictos, especialmente al acceder a recursos compartidos. Los mecanismos de IPC a menudo incluyen herramientas de sincronización para asegurar que las operaciones se realicen en el orden correcto y sin interferencia.

Los sistemas operativos modernos ofrecen una variedad de mecanismos de IPC, desde los más básicos como las **tuberías (*pipes*)** y las **señales**, hasta los más sofisticados como los **sockets**, la **memoria compartida** y el **pasaje de mensajes**. La elección del mecanismo adecuado depende de las necesidades específicas de la aplicación, como la cantidad de datos a transferir, la velocidad requerida y si la comunicación es entre procesos en la misma máquina o a través de una red.

3.2 IPC a través de tuberías

Un **pipe** (tubería) es un mecanismo de comunicación entre procesos (IPC) que permite que un proceso **envíe datos a otro**. Se comporta como un canal unidireccional, donde la salida de un proceso se convierte en la entrada de otro.

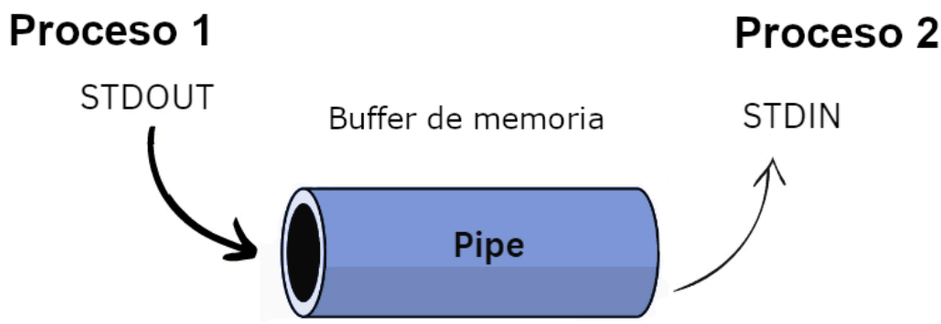
¿Cómo funcionan los pipes?

Los pipes se implementan en el sistema operativo como una sección de memoria compartida. Esta región de la memoria actúa como un búfer temporal donde un proceso puede escribir datos y otro proceso puede leerlos.

Veamos las características de los pipes:

- **Unidireccionalidad:** Significa que los datos en un *pipe* solo pueden fluir en una dirección. Imagina un caño de agua: el agua solo puede ir de un extremo al otro. En un pipe, un proceso escribe datos en un extremo (el **extremo de escritura** o el **productor**), y otro proceso los lee desde el otro extremo (el **extremo de lectura** o el **consumidor**). No hay posibilidad de que los datos regresen en la dirección opuesta a través del mismo pipe. Para la comunicación bidireccional, se necesitan dos pipes, uno para cada dirección.
- **Modelo FIFO (First-In, First-Out):** Significa que el primer dato que entra al pipe es el primero que sale. Piensa en una fila de personas esperando. La primera persona que se pone en la fila es la primera en ser atendida. Los pipes operan de la misma manera: los datos se leen en el mismo orden exacto en que fueron escritos. Esto garantiza que la información se procese secuencialmente y no se pierda el orden lógico.
- **Comunicación entre procesos relacionados:** Los **pipes anónimos** son la forma más común de usar pipes, pero tienen una limitación clave: solo permiten la comunicación entre procesos que tienen un origen común, como un **proceso padre** y su **proceso hijo**. Esto se debe a que el pipe se crea en la memoria del proceso padre antes de que se cree el hijo. Cuando el padre "bifurca" (o *forks* en inglés) un nuevo proceso hijo, el hijo hereda una copia de los descriptores de archivo del pipe de su padre, lo que les permite comunicarse a través del mismo canal. Los procesos no relacionados no tienen forma de compartir estos descriptores de archivo y, por lo tanto, no pueden usar pipes anónimos.
- **Pipes con Nombre (FIFOs):** A diferencia de los pipes anónimos, los **pipes con nombre** son archivos especiales que existen en el sistema de archivos. Esto les da una gran ventaja: **permiten la comunicación entre procesos que no están relacionados**. Un pipe con nombre se crea con un nombre de archivo, y cualquier proceso en el sistema puede abrir ese archivo para escribir o leer, tal como lo haría con un archivo regular. Un proceso puede escribir datos en el pipe, y otro proceso completamente diferente, incluso iniciado en otro momento, puede leerlos. Esto facilita la colaboración entre programas que no tienen una relación de padre e hijo.

Observe el siguiente esquema de un **pipe** (tubería):



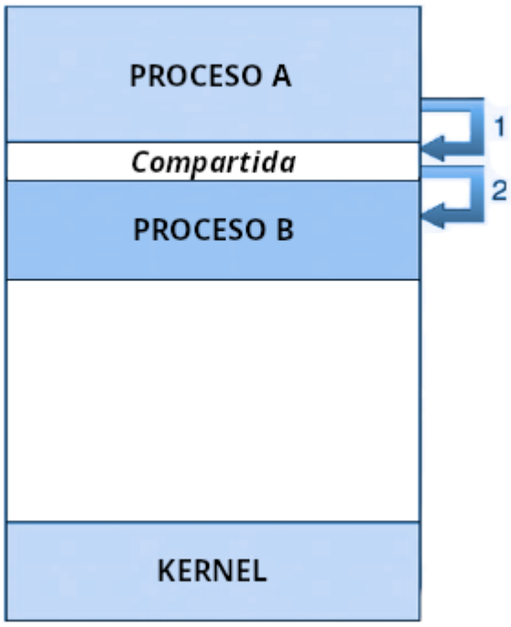
El esquema muestra tres elementos principales que trabajan juntos para permitir la comunicación entre los procesos:

- **Proceso 1 (Productor):** Este es el proceso que genera datos. Su **salida estándar (STDOUT)**, que normalmente se mostraría en la pantalla, es redirigida al pipe. Piensa en el Proceso 1 como el "productor" de la información.
- **Proceso 2 (Consumidor):** Este es el proceso que recibe y procesa los datos. Su **entrada estándar (STDIN)**, que normalmente tomaría datos del teclado, es alimentada por el pipe. Este proceso actúa como el "consumidor".
- **Pipe (Tubería):** Representado como un tubo, el pipe es el canal de comunicación unidireccional que conecta a ambos procesos. Internamente, se implementa como un **buffer en memoria** que es gestionado por el núcleo del sistema operativo. Los datos fluyen en un único sentido: del Proceso 1 al Proceso 2.

3.3 Memoria compartida (Shared Memory)

La **memoria compartida** es un mecanismo de IPC rápido donde el sistema operativo asigna un segmento de memoria para que múltiples procesos puedan leer y escribir en él directamente, evitando copias de datos y logrando alta velocidad.

Veamos cómo funciona la memoria compartida analizando el diagrama:



IPC - Diagrama de memoria compartida

1. **Mapeo de la memoria:** El **Proceso A** y el **Proceso B** no pueden acceder directamente a la memoria del otro. Para comunicarse, uno de los procesos (o ambos, o el kernel a petición de ellos) crea una región de memoria que ambos pueden "mapear" en sus propios espacios de direcciones. Esta región es la banda horizontal que se muestra como **"compartida (shared)"**.

2. **Comunicación directa:** Una vez que ambos procesos tienen acceso a esta región compartida, pueden leer y escribir datos directamente en ella. La flecha '1' representa al *Proceso A* escribiendo o leyendo datos, y la flecha '2' muestra al *Proceso B* haciendo lo mismo.

A diferencia del pasaje de mensaje, donde la comunicación requiere la intervención constante del kernel para copiar datos, en la memoria compartida, el kernel es menos utilizado. Después de eso, los procesos se comunican de forma **directa** y **rápida**, lo que hace que este método sea ideal para grandes volúmenes de datos.

Ventajas:

- **Velocidad**: Una vez que la memoria se ha compartido, los procesos pueden leer y escribir datos directamente, sin la necesidad de la intervención del kernel. Esto la hace extremadamente rápida para el intercambio de grandes cantidades de datos.
- **Eficiencia**: Se eliminan las copias de datos innecesarias, lo que mejora el rendimiento.

Desventajas:

- **Sincronización**: Los procesos deben tener mecanismos de sincronización para evitar que escriban en la misma ubicación de memoria al mismo tiempo, lo que podría causar una **condición de carrera** (race condition).
- **Complejidad**: Es más difícil de implementar que el pasaje de mensaje, ya que los programadores deben gestionar explícitamente la sincronización y el acceso concurrente a los datos.
- **Seguridad**: Si un proceso comete un error, puede sobrescribir datos de otro proceso, ya que no hay un intermediario que supervise la comunicación.

3.4 Pasaje de mensajes (message passing)

Este método de comunicación entre procesos utiliza dos primitivas (send y receive) que son llamadas al sistema (syscall) en vez de construcciones del lenguaje. Como tales, se pueden colocar con facilidad en procedimientos de biblioteca, como ejemplo:

send(destino, &mensaje);

y

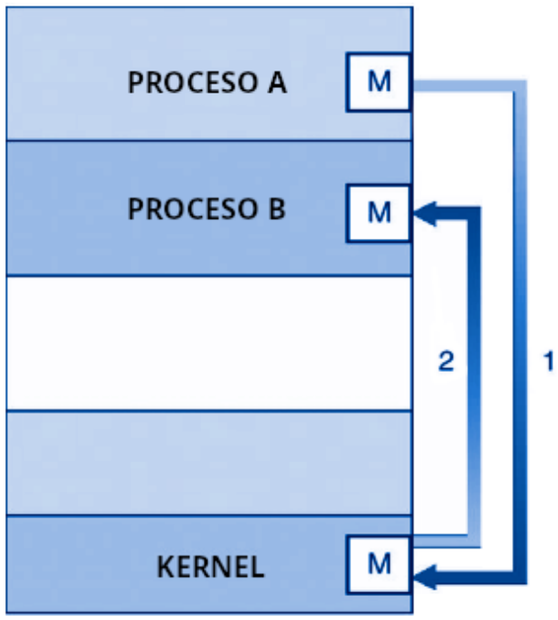
receive(origen, &mensaje);

La primera llamada envía un mensaje a un destino especificado y la segunda recibe un mensaje de un origen especificado (o de CUALQUIERA, si al receptor no le importa). Si no hay un mensaje disponible, el receptor se puede bloquear hasta que llegue uno. De manera alternativa, puede regresar de inmediato con un código de error.[1]

Este mecanismo se diferencia de *memoria compartida (shared memory)*, ya que permite a los procesos intercambiar información y sincronizarse sin compartir un espacio de memoria común.

IPC - Diagrama de memoria compartida

[1] Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición



Veamos qué indica el diagrama:

- **Proceso A y Proceso B** son dos procesos de usuario separados, cada uno con su propio espacio de memoria.
- **Kernel** es el núcleo del sistema operativo, el cual administra los recursos del sistema y provee servicios a los procesos.
- **M** es el **buffer de mensajes**, una estructura de datos temporal donde se almacenan los mensajes.

El proceso de comunicación, como se muestra en los pasos 1 y 2, funciona de la siguiente manera:

1. **Envío del mensaje:** El **Proceso A** desea comunicarse con el **Proceso B**. Para hacerlo, el Proceso A ejecuta una llamada al sistema (syscall) para enviar un mensaje. Esto provoca una transición al modo kernel. El mensaje es copiado desde el espacio de memoria del Proceso A al búfer de mensajes del kernel. La flecha etiquetada con un '1' representa este paso.
2. **Recepción del mensaje:** El kernel notifica al **Proceso B** que hay un mensaje disponible para él. El Proceso B, a su vez, realiza una llamada al sistema para recibir el mensaje. El kernel copia el mensaje desde su búfer al espacio de memoria del Proceso B. La flecha etiquetada con un '2' ilustra esta acción.

En este modelo, el kernel actúa como un intermediario o un cartero, facilitando la transferencia de información de manera segura y controlada entre procesos, ya que los procesos de usuario no tienen acceso directo a la memoria de otros.

Ventajas:

- **Simplicidad**: El modelo es más sencillo de implementar y entender.
- **Seguridad**: El kernel actúa como intermediario, lo que previene que un proceso acceda a la memoria de otro de forma accidental, evitando errores y conflictos.
- **Flexibilidad**: Funciona bien tanto para procesos en el mismo ordenador como en sistemas distribuidos.

Desventajas:

- **Lentitud**: Cada mensaje requiere que el kernel copie los datos desde el espacio de memoria de un proceso al buffer del kernel, y luego al espacio de memoria del otro proceso. Esta intervención constante del kernel y las copias de datos pueden ser lentas, especialmente con grandes volúmenes de información.

3.5 Cuadro comparativo entre los mecanismos pasaje de mensaje y memoria compartida

Característica	Pasaje de Mensaje	Memoria Compartida
Velocidad	Lento (intervención del kernel)	Rápido (acceso directo)
Sincronización	Manejada por el kernel	Requiere mecanismos externos
Seguridad	Alta (aislamiento)	Baja (sin aislamiento)
Implementación	Más sencilla	Más compleja
Uso Ideal	Cantidades pequeñas de datos, sistemas distribuidos	Grandes volúmenes de datos

Otros mecanismos de comunicación entre procesos incluyen sockets, RPC (llamadas a procedimientos remotos) y buzones.

4. Cooperación e interferencia entre procesos

La cooperación y la interferencia son dos conceptos opuestos que describen cómo interactúan los procesos al acceder a recursos compartidos.

4.1 Cooperación entre procesos

La **cooperación** ocurre cuando los procesos trabajan juntos para lograr un objetivo común. Esto requiere que se comuniquen y coordinen sus acciones. Por

ejemplo, en un sistema de tuberías, un *proceso productor* escribe datos que un *proceso consumidor* lee. Los mecanismos de comunicación entre procesos (IPC) como la memoria compartida o el paso de mensajes facilitan esta cooperación.

4.2 Interferencia entre procesos

La **interferencia** sucede cuando un proceso afecta la ejecución o los datos de otro proceso de forma no intencionada, generalmente por un acceso no controlado a un recurso compartido. Un ejemplo clásico es una **condición de carrera**, donde dos o más procesos intentan modificar el mismo dato al mismo tiempo, lo que puede llevar a resultados impredecibles o incorrectos. Los mecanismos de sincronización, son necesarios para prevenir este tipo de problemas. Estos temas se desarrollarán más adelante.



Actividad S4-A1

Realice la actividad identificada como *S4-A1* (Semana 4, Actividad 1) en la sección de actividades del campus.